

M I C R O S E R V I C E S F U N D A M E N T A L S

Load Balancing

A complete conceptual guide

What it is

How it works

Strategies

Key concepts

What is a Load Balancer?

A Load Balancer is a component that sits between clients and servers, receiving all incoming requests and intelligently distributing them across multiple server instances — ensuring no single server becomes overwhelmed.



Traffic Distributor

Routes each request to one of the available server instances based on a chosen strategy.



Health Monitor

Continuously checks which servers are alive and removes unhealthy ones from rotation.



Single Entry Point

Clients always talk to one address. The complexity of the server pool is hidden behind it.

Why Do We Need Load Balancing?

01 High Availability

If one server crashes, traffic is automatically rerouted to healthy servers. Users never see an outage.

02 Scalability

Add more server instances to handle more traffic without changing client configuration or DNS.

03 Performance

Requests are spread across servers, so no single machine gets hot while others sit idle.

04 Maintenance Windows

Take a server offline for updates — the LB drains its connections and removes it gracefully.

05 Geographic Routing

Advanced LBs route users to the nearest data center, reducing latency across regions.

06 SSL Termination

The LB handles HTTPS encryption/decryption, offloading CPU work from application servers.

How Does a Load Balancer Work?

1

Client sends request

The browser or API client sends a request to a single IP address — the load balancer's address.

2

LB receives the request

The load balancer accepts the TCP connection. The client never speaks directly to a server.

3

Algorithm selects a server

The LB consults its routing algorithm (Round Robin, Least Connections, etc.) and picks a healthy instance.

4

Request is forwarded

The LB proxies the request to the chosen server, optionally rewriting headers or injecting metadata.

5

Server responds

The server processes the request and returns the response to the load balancer.

6

Response returned to client

The LB forwards the response back. From the client's perspective, it talked to one single address.

Load Balancing Strategies (Algorithms)



Round Robin

How: Sends each new request to the next server in a circular sequence.

Best for: Best when all servers have equal capacity and request cost.



Weighted Round Robin

How: Like Round Robin, but powerful servers receive proportionally more requests.

Best for: Best when servers have different hardware specs or capacities.



Least Connections

How: Routes the request to whichever server has the fewest active connections right now.

Best for: Best for long-lived connections or variable-duration tasks.



IP Hash

How: Hashes the client's IP address to always map it to the same server.

Best for: Best when session state is stored on the server (sticky sessions).



Random

How: Picks a server at random from the healthy pool for every request.

Best for: Good for stateless workloads with uniform request costs.



Resource Based

How: The LB queries each server's actual CPU and memory usage before routing.

Best for: Best for compute-heavy workloads where utilization varies widely.

Types of Load Balancers

By OSI Layer

Layer 4 (Transport Layer)

- › Operates on TCP/UDP — sees IP addresses and port numbers only
- › Does NOT inspect the content of the request
- › Very fast and low overhead — just routes packets
- › Cannot make routing decisions based on URLs, cookies, or headers

Layer 7 (Application Layer)

- › Operates on HTTP/HTTPS — can read full request content
- › Can route based on URL path, HTTP headers, cookies, query strings
- › Enables A/B testing, canary deployments, and content-based routing
- › Slightly more overhead, but far more powerful and flexible

By Placement

Hardware LB

Dedicated physical appliance (F5, Citrix). Expensive but ultra-fast.

Software LB

Runs as software on commodity servers (Nginx, HAProxy).

Cloud LB

Managed service (AWS ALB, GCP LB, Azure LB). No infra to manage.

Client-Side LB

The client itself picks a server via service registry (gRPC, Ribbon).

How Load Balancers Detect Failures

A load balancer never blindly sends traffic. It continuously probes each server to decide who is healthy enough to receive requests.



Active Health Checks

The load balancer periodically sends its own probe requests to each server (e.g., every 5 seconds) even when no real traffic exists. If the server fails to respond — or returns an error — it is marked as unhealthy and removed from the pool until it recovers.

Examples:

- › HTTP GET /health → expects 200 OK
- › TCP connect → expects connection accepted
- › ICMP ping → expects reply within timeout



Passive Health Checks

The LB monitors real traffic responses. If a server returns too many errors (e.g., 3 consecutive 5xx responses within 10 seconds), it is temporarily removed. No extra probe traffic is sent — the observation comes from real requests.

Examples:

- › max_fails=3 → remove after 3 failures
- › fail_timeout=30s → stay removed for 30 sec
- › Automatically re-added once timeout expires

Essential Terminology

Upstream Pool

The group of server instances behind the load balancer that can receive traffic. Also called a "backend pool" or "server farm".

Sticky Sessions

A mechanism where the same client is always routed to the same server. Required when session state is stored locally on the server.

Session Persistence

The broader concept of ensuring consistent routing for a client. Can be achieved via IP Hash, cookies, or JWT tokens.

SSL Termination

The LB decrypts HTTPS traffic and forwards plain HTTP to backend servers, centralizing certificate management.

Connection Draining

When a server is removed, the LB lets its in-flight requests finish before fully cutting it off. Prevents abrupt disconnections.

Horizontal Scaling

Adding more server instances to handle increased load. Load balancing is the enabler — without it, you can't route traffic to new instances.

Load Balancer vs Reverse Proxy vs API Gateway

These three components are often confused — here is how they differ:

Feature	Load Balancer	Reverse Proxy	API Gateway
Primary role	Distribute traffic across servers	Forward requests on behalf of client	Manage, secure & route APIs
Traffic routing	Based on algorithms (RR, LC...)	Based on rules / location blocks	Based on URL, version, auth tokens
SSL termination	Yes	Yes	Yes
Auth & rate limiting	No	Limited	Yes — core feature
Health checks	Yes — built-in	Optional	Usually yes
Typical tool	Nginx, AWS ALB, HAProxy	Nginx, Caddy, Traefik	Kong, AWS API GW, Apigee

S U M M A R Y

What You Now Know



A load balancer distributes traffic, monitors health, and provides a single entry point



It works by intercepting every request and routing it via an algorithm (RR, LC, IP Hash...)



Each algorithm has trade-offs — choose based on request type and server uniformity



Health checks (active & passive) keep unhealthy servers out of the pool automatically



Layer 4 is fast but blind; Layer 7 is smart — can route based on URLs and headers



Key concepts: upstream pool, sticky sessions, SSL termination, connection draining